

AI 엔지니어링 판단력

프로덕션에서 매일 마주하는 40가지 선택의 힘

AI 엔지니어링 판단력

프로덕션에서 매일 마주하는 40가지 선택의 힘

ThakiCloud (Hyojung Han)

Contents

1	모델을 고르는 기준: 비용과 품질의 절충점	1
2	컨텍스트를 다루는 기술: 무한한 창의력, 유한한 윈도우	5
3	실패는 옵션이다 - 에러 복구 파이프라인 설계	9
4	비용을 절감하는 엔지니어링: 청구서 이해하기	13

1 모델을 고르는 기준: 비용과 품질의 절충점

LLM을 프로덕션에 처음 올리면 모든 요청에 가장 강력한 모델을 쓰고 싶은 압력이 온다. 성능 테스트에서 최고 성적을 냈기 때문이다. 하지만 청구서가 날아오는 순간 그 선택을 다시 따져보게 된다. 이 장은 모델 선택을 감이 아니라 명확한 기준으로 내리는 프레임워크를 다룬다.

1.1 비용 대 품질 매트릭스

모든 요청에 같은 모델을 쓰는 것은 비용 최적화를 포기하는 셈이다. 실제 프로덕션 워크로드에서 요청은 대체로 세 가지 유형으로 나뉜다.

유형	복잡도	모델 예	응답 시간	비용 비율
라우팅 판단	낮음	Haiku	~200ms	1x
컨텍스트 조립	중간	Sonnet	~800ms	4x
복잡 추론	높음	Opus	~3s+	19x

이 구분을 인식하면 비용을 60% 이상 줄이면서도 핵심 품질에 미치는 영향은 최소화할 수 있다. 실제로 대부분의 프로덕션 시스템에서 Opus 등급이 필요한 요청은 전체의 5~15%에 불과하다.

1.2 작업 복잡도 분류기

어떤 요청이 어느 tier에 속하는지 가장 간단하게 판단하려면 질문의 형태부터 본다. 다음 기준을 분류자로 쓴다.

먼저 질문이 어떤 말로 시작하는지 확인한다. “무엇이 / 언제 / 어디서”로 열리면 정보 검색이라 저가 모델로 처리한다. “왜 / 어떻게 / 어떤 trade-

off가”로 열리면 해석이 필요해 증가 모델을 쓴다. “만약 ~이라면”으로 가설을 세우거나 다단계 논리를 요구하는 질문에는 고가 모델이 필요하다.

출력 형식도 살펴야 한다. 단순 텍스트 응답은 저가 모델로 충분하지만, JSON 스키마가 들어가면 증가 모델이 필요하고, 구조화된 코드의 정확성이 걸려 있으면 고가 모델을 써야 한다.

마지막으로 응답을 누가 소비하는지 본다. 최종 사용자에게 직접 가는 응답은 품질 기준이 높아지고, 내부 백오피스 용도라면 잠깐의 품질 저하 정도는 넘어갈 수 있다.

세 기준을 겹쳐 보면 대부분의 요청을 분류할 수 있다. 분류기의 오분류는 회귀 테스트로 측정하며 계속 다듬는다.

1.3 주문형 vs 구독 모델 경제

API 호출 빈도에 따라 비용 구조가 크게 달라진다. 간단한 비교부터 해보자.

구독 모델의 비용은 월정액이다. 사용량이 급증해도 추가 청구가 오지 않는다. 반면 주문형 모델은 사용량에 비례해 과금된다. 1천 건/일 시스템에서 주문형이 더 유리하지만, 5만 건/일 이상에서는 구독 모델의 단건 비용이 급격히 떨어진다.

핵심 판단 기준은 “사용량이 급증할 때 비용이 급격히 올라가는 것이 비즈니스 리스크인지”다. 사용량이 예측 불가능한 제품이라면 주문형이 좋다. 사용량이 꾸준히 증가하는 백오피스 도구라면 구독이 유리하다.

구독 모델 선택 시 주의할 점은 “얼마만큼 사용해야 손해가 안 나는가”를 계산하는 것이다. 구독 비용을 토큰 수로 환산하면 주문형 대비 단가가 얼마인지 명확해진다. 이 임계점을 넘지 못한다면 주문형이 항상 더 저렴하다.

1.4 핀 전략과 에스컬레이션 전략

모든 요청을 분류기로 라우팅하면 때때로 잘못 분류해서 품질 문제가 발생한다. 이를 막는 안전장치가 핀 전략이다.

핀 전략은 요청에 예상 복잡도 이상의 품질 요구 플래그를 붙이는 것이다. 사용자가 품질 피드백을 보냈거나 이전 응답이 기대에 못 미쳤다면, 해당 세션은 이후 요청부터 자동으로 상위 모델로 라우팅된다. 이렇게 하면 첫 요청은 빠르게 처리하고 문제가 감지될 때만 비용을 추가로 들이는 구조가 된다.

에스컬레이션의 반대편은 강등이다. 상위 모델로 처리한 요청의 결과가 기대 이하였다면 그 요청 유형을 하위 모델로 내려보내는 실험을 설계한다. 가설은 “이 유형의 작업은 저렴한 모델로도 충분하다”이다. A/B 테스트로 검증하고 통계적으로 유의미한 차이가 없으면 영구적으로 하위 모델로 옮긴다.

핀과 에스컬레이션을 자동화하면 모델 선택이 정적이 아니라 동적으로 최적화된다. 사람의 판단이 필요한 순간은 시스템이 학습 데이터를 충분히 모을 때까지뿐이다.

1.5 실제로 마주하는 선택들

실무에서 자주 등장하는 판단 상황은 대략 세 가지로 좁혀진다.

가장 흔한 질문은 새 모델이 나왔을 때 바로 올릴 것인가이다. 여기서 기준은 단일 모델 성능이 아니라 전체 워크로드 성능이다. 기존 모델을 대체하기보다는 새 모델을 별도 엔드포인트로 배포하고 특정 유형의 요청만 그쪽으로 라우팅해 비교하는 편이 낫다. 전체 트래픽을 한 번에 전환하면 문제가 생겼을 때 원인을 찾기 어려워진다.

특정 요청만 상위 모델을 쓰게 할 것인가도 자주 나오는 질문이다. 이때

맞는 답은 “특정 요청 유형”이지 “특정 사용자”가 아니다. 사용자 등급으로 모델을 나누면 불평등 문제가 생기고 디버깅도 복잡해진다. 반면 요청 유형(복잡한 분석, 다단계 계획, 코드 생성 등)을 기준으로 분류하면 라우팅을 오래 유지할 수 있다.

여러 모델을 동시에 호출해서 더 나은 결과를 고르는 방법도 있다. 비용이 2배 넘게 들지만 품질 향상은 그만큼 선형으로 따라오지 않는다. 실무에서 더 잘 통하는 방식은 하위 모델로 먼저 처리하고 결과의 신뢰도가 낮을 때만 상위 모델을 호출하는 캐스케이드다. 전체를 병렬로 부르는 것보다 비용을 30~50%만 더 쓰면서 비슷한 효과를 얻는다.

2 컨텍스트를 다루는 기술: 무한한 창의력, 유한한 윈도우

컨텍스트 윈도우가 200K 토큰에 도달한 지금은 전체를 다 넣으면 된다는 생각이 들기 쉽다. 하지만 비용과 지연 시간은 컨텍스트 길이에 비례해 늘어난다. 128K 컨텍스트를 쓰면 Opus 응답 시간이 128K 시스템보다 2배 이상 걸리고 비용도 그만큼 커진다. 이 장에서는 제한된 자원을 가장 효과적으로 쓰는 방법을 다룬다.

2.1 컨텍스트 소진의 네 단계

컨텍스트가 부족해지는 상황은 네 단계로 나뉜다. 단계마다 대응 전략이 다르니 구분해서 볼 필요가 있다.

첫 단계는 경고다. 윈도우 사용률이 70%를 넘으면 시스템이 경고 플래그를 세우는데, 이 시점까지는 처리에 문제가 생기지 않는다. 이전 대화에서 중요하지 않은 부분을 건너내고, 시스템 프롬프트를 검토해 불필요한 지시문을 줄이면 공간을 확보할 수 있다.

다음 단계는 분할이다. 사용률이 85%를 넘으면 새 요청과 이전 대화의 핵심 일부만 골라 윈도우에 맞춘다. 전체 대화 이력을 다 넣는 대신 최근 N턴만, 또는 중요한 결정점만 선택하는 전략을 쓴다. 어떤 대화가 핵심인지는 이후 응답에서 실제로 참조되는 내용인지를 기준으로 판단하면 된다.

세 번째는 윈도우 초과, 즉 요청 자체가 컨텍스트 제한을 넘는 경우다. 이때는 요청을 나눠야 한다. 긴 문서를 요약하는 작업이라면 문서를 chunk로 쪼개 각 chunk를 먼저 요약하고, 그 결과를 종합하는 단계를 따로 둔다. 단계를 나누면 단계마다 컨텍스트가 들지만, 전체를 한 번에 넣는 것보다 총 비용이 낮아지는 경우가 많다.

마지막은 처리 불가다. chunk가 모델의 최소 처리 단위보다 커지면 아예 처리할 수 없다. 이런 상황은 설계 단계에서 막아야 한다. 입력 길이 제한을 두고, 초과하면 명확한 오류 메시지로 사용자에게 알린다.

2.2 RAG의 적용 시점

Retrieval-Augmented Generation(RAG)은 컨텍스트 문제를 푸는 강력한 도구지만 만능은 아니다. 적용 시점을 잘못 잡으면 비용만 늘고 품질은 오히려 떨어진다.

RAG가 필요한 경우는 크게 세 갈래로 나뉜다. 하나는 모델의 지식 컷오프 이후 정보가 필요한 경우다. 모델이 모르는 최신 뉴스, 실시간 재무 데이터, 개인화된 정보 등을 retrieve해서 컨텍스트에 얹어야 한다. 또 하나는 대규모 문서에서 특정 정보를 찾아야 하는 경우다. 수백 페이지짜리 문서에서 한 단락을 찾아야 할 때 모델에 전체를 넣는 건 비용 낭비고, 키워드 검색이나 벡터 검색으로 관련 부분만 추출해 컨텍스트에 넣는 쪽이 효율적이다. 마지막으로 여러 출처를 종합해야 하는 경우가 있다. 서로 다른 데이터 소스의 정보를 비교하거나 합칠 때 각 출처를 retrieve해서 문맥에 배치하면 모델이 더 정확하게 비교한다.

반대로 RAG가 필요 없는 경우도 있다. 질문이 모델의 훈련 데이터에 충분히 담긴 영역이라면, 그러니까 일반적인 프로그래밍 개념이나 역사적 사실처럼 모델이 이미 아는 내용이라면 검색은 불필요하다. 검색 결과의 신뢰도가 낮을 때도 마찬가지다. 검색 결과가 부정확하거나 관련성이 떨어지면 모델이 잘못된 정보를 그대로 쓰게 되므로, 이때는 검색 품질을 먼저 개선해야 한다. 응답 속도가 중요한 경우도 있다. 검색과 모델 추론을 직렬로 처리하면 지연이 늘어나므로, 즉시 응답이 필요한 용도라면 검색 없이 처리하는 쪽이 나을 수 있다.

2.3 멀티턴 대화의 상태 관리

멀티턴 대화가 길어질수록 컨텍스트는 빠르게 줄어든다. 이를 관리하는 패턴 세 가지가 실무에서 검증됐다.

첫 번째는 요약 기반 상태 관리다. N턴마다 대화의 핵심 내용을 요약해 컨텍스트에 남겨둔다. 전체 대화 이력을 그대로 두는 대신 “사용자는 A를 원했고, 모델은 B로 답했다. 다음은 C를 다룬다”처럼 압축하면 공간을 크게 아낄 수 있다. 요약을 얼마나 세밀하게 할지는 응답의 중요도에 따라 정한다. 기술적 결정이 담긴 대화는 상세하게, 단순한 정보 교환은 짧게 줄인다.

두 번째는 semantic caching이다. 사용자의 요청과 같은 의미의 요청이 이전에 있었는지 확인하고 그 결과를 재사용하는 방식이다. 같은 질문인지는 텍스트가 똑같은지가 아니라 의미적 유사도로 판단한다. 벡터 임베딩으로 유사도를 계산해 임계값을 넘으면 이전 응답을 그대로 돌려준다. 이렇게 하면 컨텍스트를 새로 쓰지 않아도 되니 비용과 대기 시간을 크게 줄인다.

세 번째는 turn allocation이다. 대화당 쓸 수 있는 최대 턴 수를 정해두고, 그 턴에 이르면 자연스러운 단락에서 대화를 전환한다. “지금까지 나온 내용을 바탕으로 새 대화에서 이어가겠습니다”라는 안내와 함께 핵심 내용을 다음 대화의 시스템 프롬프트에 담는다. 이렇게 하면 대화 세션마다 독립된 컨텍스트를 가지므로 비용을 예측할 수 있다.

2.4 실전 예지 케이스

컨텍스트 관리에는 자주 빠지는 함정이 몇 가지 있다.

“중요한 정보를 앞에 두면 더 잘 기억한다”는 말은 프로덕션에서는 뒤집힐 수 있다. 컨텍스트가 길어지면 모델은 앞부분과 뒷부분을 상대적으로 잘 기억하고, 중간에 끼워넣은 정보는 놓치기 쉽다. 그래서 핵심 정보는 요청의

앞이나 뒤에 두고, 덜 중요한 설명은 그 사이에 채운다.

또 다른 함정은 “컨텍스트를 많이 쓰면 품질이 반드시 오른다”는 가정이다. 실제로는 관련 없는 정보를 많이 넣을수록 모델이 노이즈에 붙잡혀 핵심을 놓치는 일이 생긴다. 필요한 만큼만 채우는 원칙이 품질과 비용 양쪽에 유리하다.

마지막으로 컨텍스트 관리는 순전히 기술적인 문제만은 아니다. 사용자에게 대화 길이에 대한 기대치를 설정해주는 것도 엔지니어링 판단의 일부다. “이 대화는 최대 20턴까지만 유지됩니다”라는 안내를 보여주면 사용자는 긴 대화를 여러 짧은 대화로 나누는 쪽으로 스스로 조정하게 된다.

3 실패는 옵션이다 - 에러 복구 파이프라인 설계

LLM API를 호출하는 코드를 작성한 순간부터 실패는 정상적인 현상이 된다. 네트워크 순단, 모델 overload, 토큰 초과, 입력 필터링, 출력 길이 제한처럼 원인은 다양하고 빈도는 낮지만 실패는 늘 일어난다. 이 장에서는 실패를 설계 안으로 어떻게 끌어들이는지 다룬다.

3.1 재시도 정책의 과학과 공학

“실패하면 다시 시도한다”는 전략은 누구나 떠올릴 수 있지만, 실제로 효과가 있는 재시도 정책을 설계하려면 정량적 분석이 필요하다.

재시도 간격을 정하는 방식은 크게 세 가지로 나뉜다. 고정 간격 재시도는 실패할 때마다 같은 간격으로 다시 시도하는 방식이다. 구현은 단순하지만 과부하 상황에서는 오히려 상황을 악화시킬 수 있다. 선형 백오프는 실패할 때마다 대기 시간을 1초, 2초, 3초 식으로 순차적으로 늘려간다. 지수적 백오프는 대기 시간을 1초, 2초, 4초, 8초, 16초처럼 지수함수적으로 늘리는 방식으로, 가장 널리 쓰인다.

지수적 백오프는 대부분의 상황에서 최적이지만 예외가 두 가지 있다. 하나는 사용자에게 즉각적인 응답이 필요한 경우다. 짧게 대기한 뒤 에러를 바로 고지하는 편이 어정쩡한 부분 실패보다 나을 수 있다. 다른 하나는 idempotent하지 않은 작업이다. 재시도가 같은 작업을 반복해서는 안 되는 경우인데, 결제 승인이나 결제처럼 멍등성이 없는 작업은 재시도 대신 다른 처리 경로를 마련해야 한다.

재시도 횟수에도 적절한 한도가 필요하다. 최대 재시도 횟수를 3회로 설정하면 reliability와 latency 사이에서 균형이 잡힌다. 3회를 넘기면

사용자가 응답을 기다리는 시간만 불필요하게 길어지고 시스템 자원도 낭비된다. 3회를 시도해도 풀리지 않는 문제는 대개 그 이상 반복해도 풀리지 않는다.

3.2 부분 실패와 전체 실패의 경계

모든 실패가 같은 무게를 갖지는 않는다. 어떤 실패는 부분적으로 수습할 수 있고, 어떤 실패는 전체를 끌고 무너진다.

부분 실패는 응답의 일부만 생성된 상태를 말한다. 모델이 생성 도중에 멈추더라도 이미 만들어진 부분에 쓸 만한 정보가 담겨 있을 수 있다. 이 부분을 사용자에게 그대로 돌려줄지는 판단이 필요한 문제다.

부분 실패를 허용할지는 세 가지 기준으로 판단한다. 먼저 생성된 부분이 그 자체로 의미를 갖는지 봐야 한다. 응답의 앞부분만 만들어지고 뒷부분이 빠졌더라도 앞부분이 완결된 문장이나 단락으로 끝난다면 부분 제공이 가능하다. 다음으로는 부분 제공이 사용자를 혼란스럽게 만들지 않는지 살펴야 한다. “이 답변은 중도에 중단되었습니다”라는 안내와 함께 부분 응답을 제공하는 편이 완전한 답변만 기다리게 하는 것보다 나을 수 있다. 마지막으로 응용 영역에서 부분 응답이 실제로 쓸모가 있는지 따져야 한다. 코드 생성이 중간에 끊기면 컴파일조차 되지 않으니 부분 제공은 의미가 없다. 반면 텍스트 요약이나 brainstorming이라면 절반짜리 통찰도 나름의 가치를 지닌다.

전체 실패는 복구가 불가능한 상태를 말한다. 토큰 limits exceeded, 모델 서비스 종료, 입력 유효성 검증 실패 등이 여기 속한다. 이때 시스템은 두 가지 원칙을 지켜야 한다. 하나는 실패를 즉시 통고하는 것이다. 모호한 에러 메시지보다 정확한 실패 이유를 밝히는 편이 신뢰를 지키는 길이다. 다른 하나는 대안 경로를 제시하는 것이다. “이 작업은 지금 처리할 수 없습니다. 나중에 다시 시도하거나 다른 방식으로 시도해주세요.”처럼 사용자가 취할 수 있는 조치를 함께 안내한다.

3.3 복구 불가능 상태의 사용자 경험

복구가 불가능한 순간, 사용자에게 남는 것은 에러 메시지뿐이다. 이때 어떤 경험을 남기느냐가 product의 성패를 가른다.

좋은 에러 메시지에는 명확성, 다음 단계 제시, 감정 배려라는 세 가지 조건이 필요하다. 명확성은 무엇이 잘못되었는지 정확히 설명하는 일이다. “문제가 발생했습니다”보다는 “AI 모델이 일시적으로 사용할 수 없습니다. 이는 일반적으로 30초 내 정상 복구됩니다.”가 훨씬 낫다. 다음 단계 제시는 사용자가 취할 수 있는 조치를 알려주는 일이다. “새로고침 버튼을 클릭하거나 잠시 후 다시 시도해주세요.”라는 문구만 있어도 사용자는 helplessness를 느끼지 않는다. 감정 배려는 에러 상황을 침착하게 설명하면서도 사용자가 불편을 겪었다는 사실을 인정하는 일이다. “불편을 드려 죄송합니다”라는 한마디가 만족도에 의외로 큰 영향을 미친다.

이 원칙을 실제로 적용한 사례도 있다. 대규모 분석 결과를 생성하는 한 도구는 분석 도중 모델 서비스가 종료되면 전체 분석을 처음부터 다시 돌리는 대신, 이미 끝난 부분을 저장해두고 이어갈 수 있는 checkpoint 기능을 구현했다. 사용자는 처음부터 다시 시작할 필요 없이 미완료 부분만 다시 처리하면 되었고, 그만큼 시간 낭비도 크게 줄었다.

3.4 Circuit Breaker 패턴의 적용

LLM API 호출에서 circuit breaker 패턴은 시스템 전체를 지켜주는 효과적인 수단이다.

동작 원리는 간단하다. 일정 기간 동안 실패율이 임계값을 넘으면 circuit breaker가 열리면서 후속 호출을 곧바로 실패 처리한다. 그래서 이미 문제가 있는 시스템에 부하가 더 쌓이지 않는다. cooldown 기간이 지나면 제한된 수의 요청만 통과시켜 문제가 풀렸는지 확인하고, 시스템이 정상적으로 동작하면 circuit breaker가 닫히면서 호출이 다시 이어진다.

LLM API에서 circuit breaker를 설정할 때는 실패 유형을 구분하는 일이 중요하다. 429(too many requests) 오류는 일시적인 과부하를 나타내므로 circuit breaker를 열 근거가 된다. 반면 401(인증 오류)이나 400(입력 오류)은 circuit breaker를 열 이유가 없다. 전자는 인증 정보를 확인하면 풀리고, 후자는 입력을 고치면 그만이다.

실제 구현에서는 circuit breaker 임계값을 정하는 경험적 기준이 있다. 실패율 50퍼센트를 임계값으로 삼고 10분을 window로 측정하는 방식이다. 10분 동안 호출이 10회 이상 있었고 그중 5회 이상 실패하면 circuit breaker가 열린다. cooldown 기간은 30초에서 1분 정도가 적당한데, 너무 짧으면 시스템이 아직 불안정한 상태에서 재시도가 몰려 추가 실패를 부르고, 너무 길면 서비스 중단이 불필요하게 늘어진다.

3.5 에러 로깅과 피드백 루프

실패 기록을 잘 설계하면 시스템을 개선할 귀중한 데이터가 된다. 그렇다면 무엇을 얼마나 기록해야 할까.

핵심 원칙은 구조적으로 기록하되 평범한 중복은 걸러내는 것이다. 실패 유형마다 건수만 기록해도 충분한 경우가 있지만, 새로운 유형의 실패가 나타나면 추가 정보를 남겨 패턴을 분석할 수 있게 해야 한다.

반복되는 특정 실패 패턴은 라우팅 전략을 바꿔 해결할 수 있다. 가령 특정 유형의 요청에서 토큰 limits exceeded 오류가 계속 발생한다면 해당 유형을 더 작은 chunk로 나누는 방법이 있다. 실패율과 무관하게 latency가 늘어나는 패턴도 경고 신호가 될 수 있다. 평소보다 높은 평균 토큰 소비가 계속되면 프롬프트가 부풀었거나 응답 길이 제한이 느슨해졌을 가능성을 의심해봐야 한다.

4 비용을 절감하는 엔지니어링: 청구서 이해하기

매달 청구서를 받으면 예상과 다른 금액에 놀라는 경우가 많다. 어떤 호출이 비용을 가장 많이 잡아먹는지, 언제 비용이 급증했는지 알려면 비용 구조를 정밀하게 뜯어봐야 한다. 이 장에서는 비용을 예측 가능한 범위로 관리하는 기술을 다룬다.

4.1 토큰 계산의 실제

LLM 비용의 기본 단위는 토큰이다. 토큰은 모델이 텍스트를 처리하는 최소 단위이며, 영어에서는 대략 4글자가 토큰 하나에 해당하고 일본어에서는 글자 단위로 쪼개지기도 한다. 비용을 계산하려면 입력 토큰 수와 출력 토큰 수를 따로 세야 하는데, 둘의 단가는 다르게 책정되는 경우가 대부분이다.

입력 토큰 수는 글자 수를 세는 간단한 식으로 대략 추정할 수 있지만, 정밀하게 계산하려면 실제 토큰화 API를 호출해야 한다. 추정 공식을 쓴다면 문장당 약 1.5 토큰을 기본값으로 잡고 기술 용어나 고유명사가 많으면 2배를 곱한다. 다만 이건 어디까지나 대략치이고, 정확한 수치는 API 응답의 usage 필드에서 확인하는 편이 믿을 만하다.

출력 토큰 수를 예측하는 일은 더 어렵다. 응답 길이는 입력의 복잡도와 원하는 상세 수준에 따라 크게 달라지기 때문이다. 가장 보수적인 방법은 출력 토큰 수에 상한을 걸고 그 이상의 응답은 잘라내는 것이다. 그러면 비용의 상한선을 미리 예측할 수 있다.

비용을 줄이는 첫 번째 원칙은 입력을 작게 유지하는 것이다. 시스템 프롬프트를 간결하게 쓰고 예시를 최소화하고 불필요한 설명을 건너내면

입력 토큰 수가 줄어든다. 다만 품질과 맞바꾸는 관계라 주의가 필요하다. 지시사항은 짧게 쓰되 핵심 요구사항만은 명확히 남겨야 한다.

두 번째 원칙은 출력을 필요한 만큼만 요청하는 것이다. `max_tokens`로 응답 길이에 제한을 걸면 비용 상한을 통제할 수 있다. 사용자에게 500단어짜리 답변이 굳이 필요 없다면 200단어로 줄여도 핵심 정보는 대부분 그대로 전달된다.

4.2 배치 처리의 전략적 활용

배치 처리는 여러 요청을 한꺼번에 처리해 비용을 크게 줄인다. 다만 trade-off가 있어서 언제 써야 하는지 정확히 판단해야 한다.

배치 처리가 힘을 발휘하는 경우는 크게 세 가지다. 하나는 실시간 응답이 필요 없는 백그라운드 작업이다. 대량 문서 일괄 분류, 로그 분석, 데이터 전처리가 여기에 해당하고, 이런 작업은 수십 초의 지연도 문제가 되지 않으므로 배치 처리가 잘 맞는다. 또 하나는 비슷한 요청을 여러 건 처리하는 경우다. 서로 다른 입력에 같은 모델과 시스템 프롬프트를 반복해서 쓰는 작업이라면 배치로 묶어 처리할 때 대기 시간과 비용을 동시에 줄일 수 있다. 마지막으로 비용 최적화 자체가 중요한 경우다. 배치 API는 단위당 비용이 평소보다 10~50% 낮아서 대량 처리에서는 그 차이가 무시할 수 없는 수준이 된다.

배치 처리의 한계도 알아둬야 한다. 배치 응답은 전체가 끝나야 한꺼번에 돌아오므로 요청 하나만 오래 걸려도 전체가 지연된다. 배치 안에서 개별 요청의 우선순위를 정할 수도 없어서, 급한 요청이 섞여 있으면 배치 처리 자체가 맞지 않는다.

실무에서 배치와 스트리밍은 이런 기준으로 나눈다. 사용자가 화면을 보며 기다리는 동기적 상호작용이라면 스트리밍이 맞다. 토큰이 생성되는 대로 바로 전송되므로 첫 토큰까지 걸리는 시간이 배치보다 짧다. 반대로

사용자가 결과를 바로 확인하지 않는 백그라운드 작업이라면 배치가 낮다. 스트리밍의 추가 비용과 complexity를 피하면서 더 낮은 단가를 누릴 수 있다.

4.3 캐싱 전략의 설계

캐싱은 반복 요청의 비용을 아예 없앨 수 있는 가장 확실한 방법이다. 다만 효과적인 캐싱 전략을 짜려면 먼저 호출 패턴을 분석해야 한다.

가장 단순한 방식은 완전 일치 캐싱이다. 입력 텍스트가 정확히 같으면 저장된 응답을 그대로 돌려준다. 영어에서는 이 방식이 잘 통하지만, 한국어처럼 띄어쓰기나 문장 부호에 따라 의미가 달라지는 언어에서는 동일 문서를 찾기가 까다롭다. 이럴 때는 의미적 캐싱을 쓴다. 입력의 벡터 임베딩을 계산해두고, 이전에 본 임베딩과의 코사인 유사도가 임계값을 넘으면 저장된 응답을 반환하는 방식이다.

의미적 캐싱에서는 임계값 설정이 관건이다. 임계값을 높게 잡으면(예: 0.95 이상) 거짓 양성률이 줄어들지만 캐시 적중률도 함께 낮아진다. 반대로 낮게 잡으면(예: 0.80) 적중률은 오르지만 잘못된 응답을 돌려줄 위험이 커진다. 권장 범위는 0.90에서 0.95 사이이고, 실제 적중률과 품질을 지켜보면서 조정하는 편이 좋다.

캐시를 제대로 관리하려면 TTL(Time To Live)을 설정해야 한다. TTL이 너무 길면 오래된 응답이 그대로 나갈 수 있고, 너무 짧으면 캐시 효과가 줄어든다. 일반적으로는 데이터 신선도가 중요한 정보에는 짧은 TTL을, 일반 지식 기반 응답에는 긴 TTL을 준다.

캐시 무효화 전략도 필요하다. 모델이 갱신되면 이전 버전의 응답은 더 이상 유효하지 않을 수 있는데, 그렇다고 모든 사용자의 세션을 한꺼번에 무효화하면 서비스가 끊길 수 있다. 새 모델 버전의 응답만 fresh로 표시하고 이전 버전 응답은 점진적으로 교체하는 점진적 무효화가

현실적인 접근이다.

4.4 비용 모니터링의 실행

비용을 줄으려면 먼저 지금 비용이 어디서 발생하는지 정량적으로 파악해야 한다. 이를 위한 모니터링 체계를 설계해보자.

비용 분석의 기본은 차원별로 비용을 추적하는 것이다. 모델별, 엔드포인트별, 사용자 세그먼트별, 시간대별로 비용을 쪼개보면 비용 대부분이 어디서 발생하는지 드러난다. 단일 모델이 전체 비용의 80%를 차지한다면 그 모델을 최적화하는 것만으로 전체 효과의 대부분을 얻을 수 있다.

비용에 이상 징후가 나타나는 패턴은 여러 가지다. 평소와 다른 시간대에 비용이 급증한다면 예정에 없던 API 호출이 발생하고 있을 가능성이 있다. 특정 사용자 세그먼트에서만 비용이 늘었다면 그 세그먼트에만 영향을 준 변화가 있었을 수 있다. 평균 토큰 소비가 평소보다 높은 채로 이어진다면 프롬프트가 부풀었거나 응답 길이 제한이 느슨해졌을 가능성이 있다.

비용 알림의 임계값을 정하는 것도 중요하다. 일일 예산의 80%에 도달하면 경고를 보내고 100%에 도달하면 자동으로 차단하는 것이 일반적인 전략이다. 이렇게 해두면 예기치 못한 비용 급증을 막을 수 있다.

비용 관리는 결국 기술적 노력과 business metric 사이의 균형 문제다. 캐싱과 배치 처리를 극대화하면 비용은 줄어들지만 일부 기능에서는 응답 시간이나 품질이 떨어질 수 있다. 최종 목표는 비즈니스 가치를 최대화하는 것이지 단순히 비용을 최소화하는 것이 아니다. 비용 최적화는 기술적으로 가능한 것과 business에 미치는 영향을 함께 저울질해서 결정해야 한다.